TPDS-III

LAB ASSIGNMENT BASED ON MODULE 3 and 4

Submission Deadline: 25th October 2026

Module 3

Practical 1: Student Management REST API Using Node.js, Express.js and MySQL

Recommended for HTTP and CRUD Operations

Aim

Design and develop a RESTful backend application for managing student academic records using Node.js, Express.js, and MySQL.

Problem Statement

Develop a Student Management System that allows an administrator to add, view, update, delete, and search student records through REST API endpoints.

Functional Requirements

1. Create an Express.js web server running on port 3000.
2. Connect the application to a MySQL database.
3. Create a students table containing Student ID, Name, Email, Department, Semester, and CGPA.
4. Implement CRUD operations using HTTP methods GET, POST, PUT, and DELETE.
5. Search student records by department.
6. Validate required fields and prevent duplicate email addresses.
7. Return appropriate JSON responses and HTTP status codes.
8. Test APIs using Postman.

.

Practical 2: Online Course Enrollment and Management System Using Node.js, Express.js and SQL Server

Recommended for Database Relationships and Backend Logic

Aim

Develop a backend application that manages course registration, student enrollment, and seat availability using Node.js, Express.js, and Microsoft SQL Server.

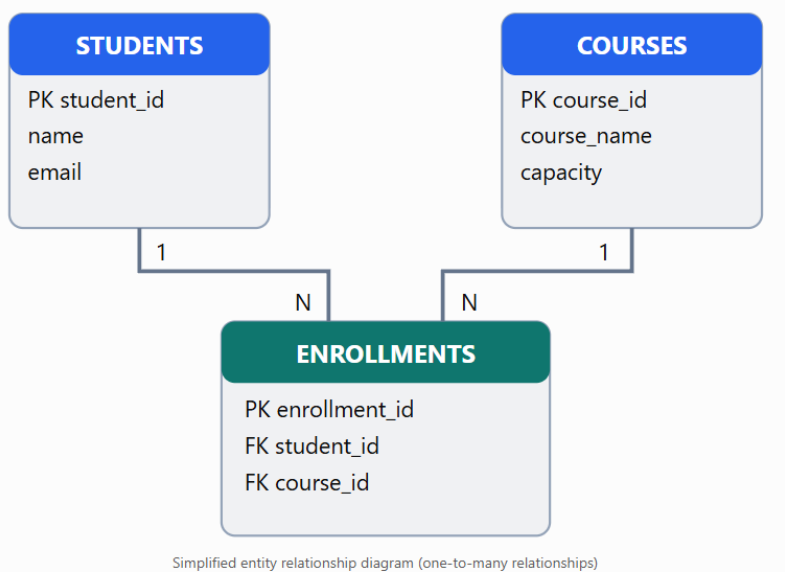
Problem Statement

A college requires an Online Course Enrollment System where students can browse courses, enroll in available courses, and view their registered courses. The system should prevent duplicate enrollment and registration beyond course capacity.

Functional Requirements

1. Create an Express.js server.
2. Connect Node.js to Microsoft SQL Server using the mssql package.
3. Create three relational tables: Students, Courses, and Enrollments.
4. Display available courses and their remaining seats.
5. Allow students to enroll in courses using a POST request.
6. Prevent duplicate enrollment.
7. Prevent enrollment when a course is full.
8. Allow students to cancel enrollment.
9. Display all courses enrolled by a particular student.
10. Use SQL joins and transactions to maintain data consistency.

Database Design



Simplified entity relationship diagram (one-to-many relationships)

Module 4

Practical 1: Website Development and Version Control Using Git and GitHub

Git and GitHub

Aim

Design and develop a responsive personal portfolio website and manage its source code using Git and GitHub, demonstrating repository creation, branching, commits, merging, and push/pull operations.

Problem Statement

Develop a responsive portfolio website containing Home, About, Skills, Projects, and Contact sections. Use Git for version control and GitHub for remote repository management. Demonstrate collaborative development through branches and pull requests.

Software Requirements

- Visual Studio Code
- Git and GitHub
- HTML5, CSS3, Bootstrap 5, JavaScript
- Web browser

Functional Requirements

1. Develop a responsive personal portfolio website using HTML, CSS, Bootstrap, and JavaScript.
2. Include navigation, profile, skills, projects, and contact sections.
3. Initialize a local Git repository using `git init`.
4. Add files and create commits using `git add` and `git commit`.
5. Create a separate feature branch and implement a new feature.
6. Merge the feature branch into the main branch using a pull request.
7. Push the source code to GitHub.
8. Clone the repository and demonstrate `git pull`.
9. Display commit history using `git log`.
10. Deploy the website using GitHub Pages.

Practical 2: Dockerization and Cloud Deployment of a Node.js Web Application

Docker and Cloud Deployment

Aim

Develop a Node.js and Express.js web application, containerize it using Docker, and deploy it to a cloud platform.

Problem Statement

Develop a Student Task Management Web Application that allows students to create, view, update, and delete tasks. Containerize the application using Docker and deploy it to a cloud hosting platform.

Software Requirements

- Node.js and Express.js
- HTML, CSS, Bootstrap
- Docker Desktop
- Git and GitHub
- Render, Azure, AWS, or another Docker-compatible cloud platform

Functional Requirements

1. Develop a responsive Task Management application.
2. Create a Node.js server using Express.js.
3. Implement REST API endpoints for CRUD operations.
4. Store tasks using a database or persistent storage.
5. Create a Dockerfile for the application.
6. Build a Docker image using docker build.
7. Run the application inside a Docker container.
8. Test the application locally through port mapping.
9. Push the source code to GitHub.
10. Deploy the Dockerized application on a cloud platform and verify its public URL.

